MLDS 490 — Assignment 2

Deep Q-Network on CartPole-v1 and MsPacman-v5

Weicheng · GitHub: weicheng998

November 11, 2026

Submission information

GitHub username	`weicheng998`
Repository	`mlds490-assignment2` — https://github.com/weicheng998/mlds490-assignment2
Code	everything is in `dqn/` · launch scripts: `scripts/` · logs and figures: `results/`
Compute	deepdish3, one RTX 2080 Ti per run (`CUDA_VISIBLE_DEVICES`), conda env `myenv`, PyTorch 2.14 cu126, `gymnasium[atari]==1.0.0`, `ale-py==0.10.1`. CPU-only validation scripts ran on my laptop

How to run (full details in `README.md`):

```
pip install -r requirements.txt
nohup bash scripts/run_cartpole.sh 0        > logs/cartpole.log        2>&1 &  # 5 seeds + tests
nohup bash scripts/run_mspacman.sh 1 0 1    > logs/mspacman_s0.log     2>&1 &  # Double DQN, see
nohup bash scripts/run_mspacman.sh 2 1 1    > logs/mspacman_s1.log     2>&1 &  # Double DQN, see
nohup bash scripts/run_mspacman.sh 3 0 0    > logs/mspacman_s0_dqn.log 2>&1 &  # vanilla DQN
bash scripts/eval_mspacman.sh 1                         # 500 test episodes per run, random baseli
python -m dqn.validate --allow-cpu                      # validation suite (CPU, about 6 minutes)
python -m dqn.validate --allow-cpu --tests pacman_env
python -m dqn.ablate --group ablation --seeds 5 --jobs 9 --allow-cpu
python -m dqn.ablate --group gamma    --seeds 3 --jobs 9 --allow-cpu
python -m dqn.plot && bash report/build.sh              # figures, summary.json, this PDF
```

Training and evaluation require CUDA and stop with an error at startup if it is missing. `--allow-cpu` exists only for smoke tests and the CPU-only validation scripts. Each run uses a single GPU. Runs checkpoint atomically and resume when the same command is started again.

Where the numbers come from. Every number in this report is read from a log or result file in `results/`. The figures are produced by `dqn/plot.py`, and the numbers in the text are filled in from `results/summary.json` by `report/build.py`, which refuses to build if a value is missing.

How validation was approached. As in Assignment 1, I wrote down the expected outcome before running each test, and I compared against something that does not share code with the agent wherever I could: a closed form, exact value iteration, an independent numpy computation, or a reference

distribution. Five of my predictions were wrong or only partly right. They are marked where they occur (Sections 2.4 and 4), together with what they showed.

1 The algorithm (shared by both games)

One implementation (`dqn/agent.py`, `dqn/replay.py`, `dqn/train.py`) is used for both games. Only the network and the hyperparameters differ.

Update rule. A transition $(s, a, r, s', \text{cut})$ is drawn uniformly from the replay buffer and the online network Q_θ is regressed on

$$y \;=\; r + \gamma\,(1 - \text{cut})\; Q_{\bar\theta}\big(s',\, \arg\max_{a'} Q_\theta(s', a')\big),$$

with the Huber loss between $Q_\theta(s, a)$ and y. $Q_{\bar\theta}$ is the **target network**, a copy of the online network that is refreshed every fixed number of gradient steps (hard sync). The online network chooses the next action and the target network evaluates it (Double DQN); `--double 0` gives the original $\max_{a'} Q_{\bar\theta}(s', a')$. The behaviour policy is ε-greedy with a piecewise-linear ε schedule, and the learning rate decays linearly.

Terminal versus truncation. `cut` is 1 only when the episode really ended at s': the pole fell or the cart left the track in CartPole; a life was lost or the game ended in MsPacman. When an episode is stopped by a time limit (500 steps in CartPole-v1, 108,000 emulator frames in MsPacman-v5) the task itself has not ended, so `cut` stays 0, the true next observation is stored, and the target still bootstraps from it. Section 4 shows what happens when this is done wrong.

Hyperparameters.

	CartPole-v1	MsPacman-v5
Discount γ	0.95	0.99
Network	MLP 4–128–128–2	conv 32@8×8/4, 64@4×4/2, 64@3×3/1, FC 512, 9 outputs
Input	the 4 state variables	4 stacked frames from the handout `preprocess_observation` (88×80, int8), divided by 128
Parallel environments	1	8 (worker processes), one GPU
Replay buffer	200,000 transitions (whole run)	500,000 transitions
Batch · update frequency	128 · every step	32 · one update per 4 agent steps
Optimizer	Adam, lr $10^{-3} \to 10^{-5}$ linear	Adam, lr $10^{-4} \to 2.5 \times 10^{-5}$ linear, $\epsilon_{\text{Adam}} = 10^{-5}$
Loss · gradient clipping	Huber · norm 10	Huber · norm 10
Target network	hard sync every 500 updates	hard sync every 2,500 updates (10,000 agent steps)
ε schedule	$1 \to 0.05$ over 10k steps, $\to$ 0.01 at 50k	$1 \to 0.1$ over 1M steps, $\to 0.01$ at 4M
Warm-up before learning	1,000 buffer slots	50,000 buffer slots
Training reward	raw (+1 per step)	$\text{sign}(r) \in \{0, 1\}$ (clipped)
Bootstrap cut	termination	termination or life lost

	CartPole-v1	MsPacman-v5
Training length	200,000 steps	1,035,984 agent steps (1,737 episodes)
Periodic validation	100 greedy episodes every 5,000 steps	30 greedy episodes every 250,000 steps

Reward scaling and units. CartPole rewards are used as they are, so CartPole Q-values are in "steps": the true value can never exceed $\sum_k \gamma^k = 1/(1-\gamma) = 20$. MsPacman rewards are clipped to their sign for training, so MsPacman Q-values are in *clipped-reward units* (roughly "discounted number of reward events before the next lost life"), with a ceiling of $1/(1-\gamma) = 100$. Every reward plot, histogram and score in this report is the **raw game score**.

Definition of "maximum Q-value". For plot (i) I log, for every training episode, the maximum over the states visited in that episode of $\max_a Q_\theta(s, a)$, computed with the online network at the moment the state was visited. This number depends on which states the episode happened to visit, so I also log the mean of $\max_a Q_\theta(s, a)$ over a *fixed* set of held-out states (1,000 for CartPole, 2,048 for MsPacman, collected once with a random policy on seeds that are never used for training). The fixed-state curve only changes because the network changes. Both are in the plots.

Seeds and leakage. Every episode has an explicit seed. Training episodes use seeds below 10^8, periodic validation uses $2 \cdot 10^9 + i$ and the final test uses $10^9 + i$. `best.pt` is the checkpoint with the best validation score (ties go to the later one); `final.pt` is the last network. The 500 test episodes are never looked at during training or selection, and I report both checkpoints.

Design decisions.

- *One code path.* CartPole is the same agent with a 1-frame "stack" and 1 environment, so the tests on CartPole and on the tabular MDP exercise the code that trains MsPacman.
- *Replay storage.* The buffer stores single frames (int8) and rebuilds 4-frame stacks when sampling, which is 4× smaller than storing stacked states. The last observation of every episode gets its own non-sampleable slot, so the next state of any transition is simply "the stack ending one slot later", also at a truncation.
- *Life loss as a bootstrap cut in MsPacman.* The game continues after a lost life, so this is a modeling choice rather than a fact about the environment. It tells the agent directly that dying is bad (otherwise the only penalty is the missing future reward several hundred steps later). The price is that Q is a "value until the next death", which is what the calibration plot in Section 3 compares against.
- *Reward clipping.* Clipping keeps the TD errors in the range where the Huber loss is well behaved and lets the CartPole learning-rate scale carry over. It also makes the agent indifferent between a 10-point pellet and a 200-point ghost; the consequence is discussed in Section 3.
- *Sticky actions and frame skip.* I kept the `v5` defaults (frame skip 4, sticky-action probability 0.25, 9 actions) because the assignment names this version. Sticky actions make the environment stochastic, which matters for the evaluation (Section 3.3).
- *Vectorized acting.* Eight environments step in lock-step and each gradient step uses fresh uniform samples, with the standard ratio of one update per four agent steps.
- *Evaluation is greedy* ($\varepsilon = 0$): "roll out using your trained neural network".

2 CartPole-v1 ($\gamma = 0.95$)

2.1 Results

All five seeds use the configuration in Section 1. Plots are shown for seed 0; the table covers every seed.

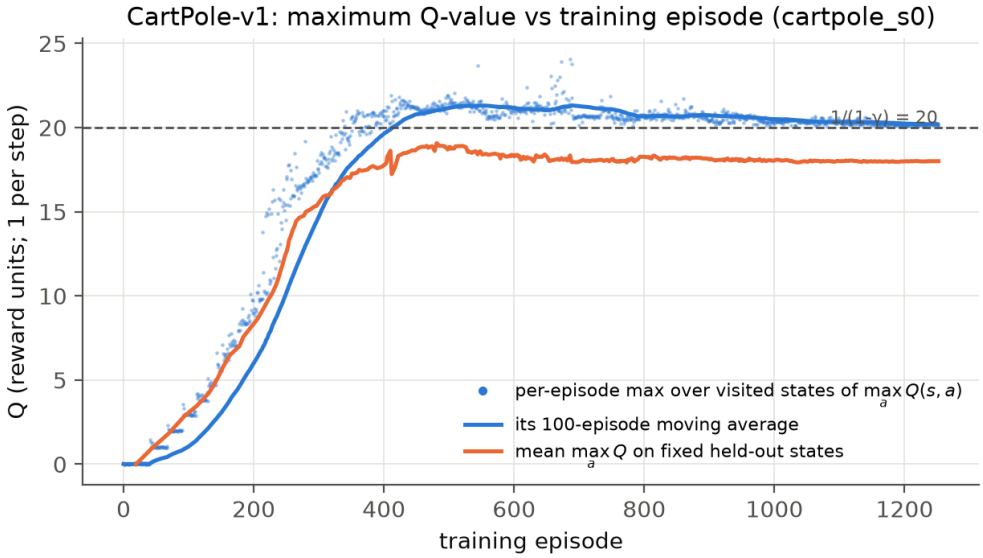


Figure 1: CartPole (i). Maximum Q-value against training episode, seed 0. Blue dots: for each training episode, the largest $\max_a Q(s, a)$ over the states it visited. Orange: mean $\max_a Q$ on 1,000 fixed held-out states. The dashed line is the largest value any policy can obtain, $1/(1 - \gamma) = 20$.

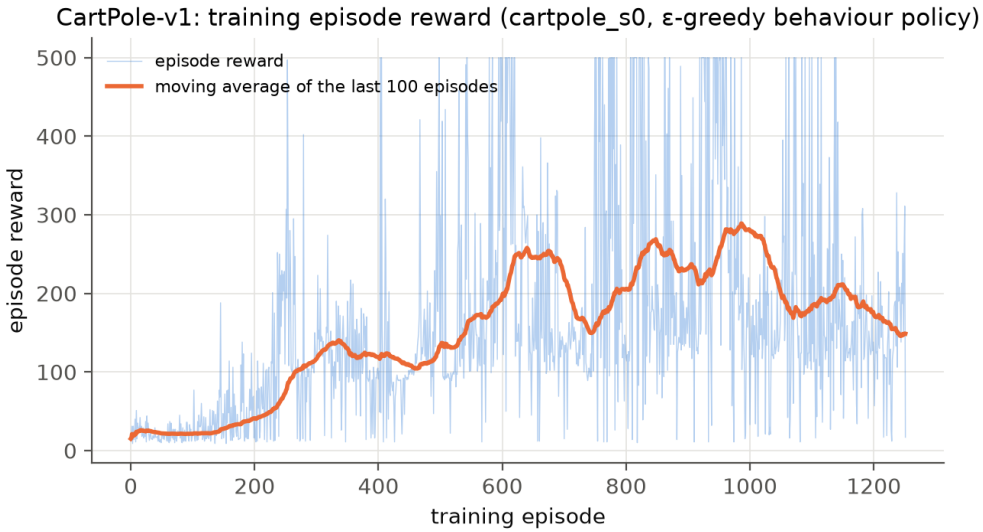


Figure 2: CartPole (ii). Training episode reward (blue) and its 100-episode moving average (orange), seed 0. These are ε-greedy training episodes.

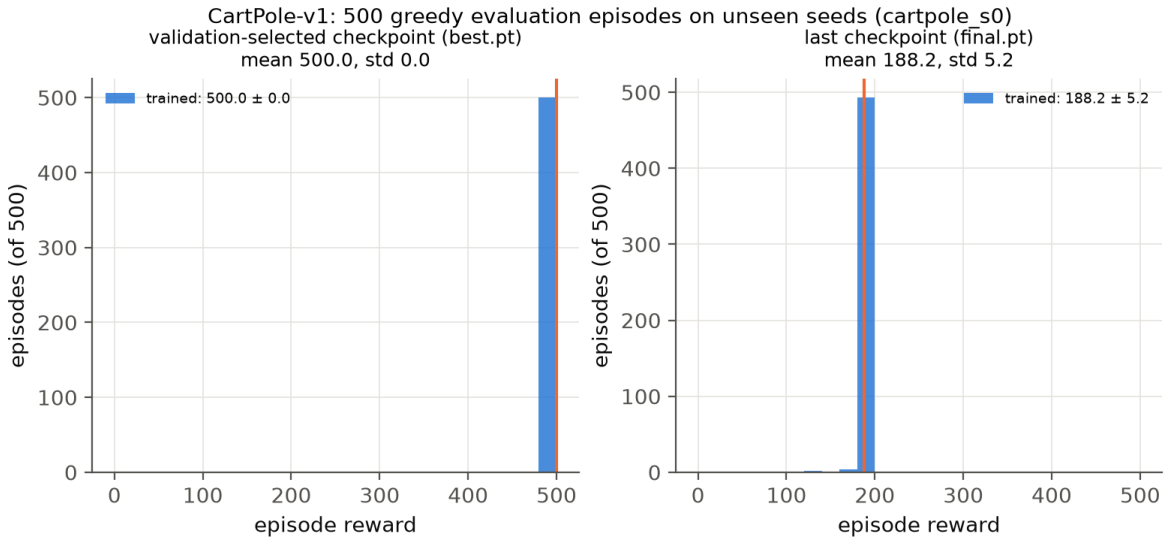


Figure 3: CartPole (iii). Histogram of 500 greedy test episodes on seeds never used in training or checkpoint selection, seed 0. Left: the validation-selected checkpoint. Right: the last network of the same run.

Seed	Episodes	First 500-step training episode	`best.pt`: mean ± std (episodes at 500)	`final.pt`: mean ± std (episodes at 500)	Random policy
0	1252	#403	**500.0 ± 0.0** (100%)	188.2 ± 5.2 (0.0%)	22.4 ± 12.7
1	1232	#554	**500.0 ± 0.0** (100%)	499.5 ± 10.6 (99.8%)	same episodes
2	1151	#591	**500.0 ± 0.0** (100%)	370.6 ± 106.9 (31.6%)	
3	1292	#589	**500.0 ± 0.0** (100%)	490.3 ± 24.7 (82.0%)	
4	1201	#480	**500.0 ± 0.0** (100%)	438.1 ± 81.7 (55.6%)	

The environment is solved, with a qualification. For every seed, the checkpoint selected on 100 validation episodes keeps the pole up for the full 500 steps in all 500 test episodes (mean 500.0, standard deviation 0.0). Against the random policy on the same 500 seeds (22.4 ± 12.7), the difference is 477.6 steps with a 95% interval of [476.5, 478.7] (Welch $t = 843$; Mann–Whitney $p = 1e - 188$; every trained episode beats every random one).

The qualification is that **training is not stable**, and I do not want the 500.0 to hide that. The last network of the same runs scores between 188 and 500 depending on the seed. In seed 0 only 10% of the 40 periodic validations reached a mean of 500, the 100-episode moving average of the training reward peaks at 289, and it never reaches 475. The agent repeatedly finds a policy that balances for 500 steps and then loses it again (Figure 4, middle). The selection uses only validation seeds, so the 500.0 is an honest estimate for *that checkpoint*; it is not an estimate of what a randomly chosen moment of training would give.

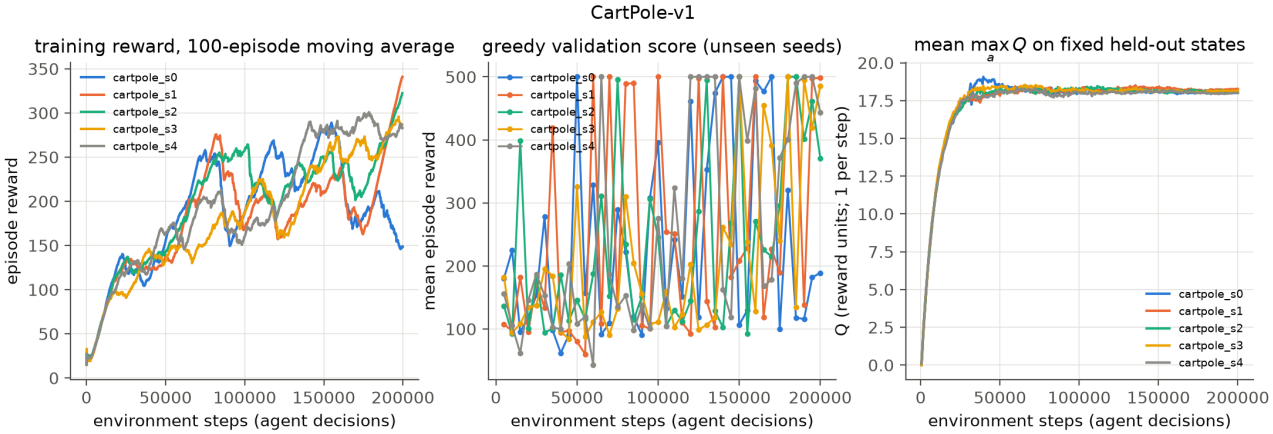


Figure 4: All five CartPole seeds against environment steps. Left: 100-episode moving average of the training reward. Middle: mean reward of 100 greedy validation episodes every 5,000 steps; each seed reaches 500 several times and falls back. Right: mean $\max_a Q$ on the held-out states settles at about 18 for every seed.

2.2 Discussion

Maximum Q. The held-out curve rises smoothly and settles at 18.0, below the ceiling of 20, which is what it should do: the held-out states come from a random policy, and many of them are a few steps from falling. The per-episode maximum behaves differently. It crosses 20 around episode 350 and stays above it: in seed 0, 72% of all training episodes contain a state whose predicted value is above 20, with a peak of 24.1. No policy can collect more than 20, so this is overestimation that can be proven without knowing Q^*. It is small (on the test rollouts of `best.pt` the mean predicted value is 20.37 against a realized 19.99, a bias of +0.38), and it shrinks as the learning rate decays (the mean over the last 100 episodes is 20.18). The two "max Q" definitions therefore tell different stories: the per-episode maximum is an upper-tail statistic and exposes overestimation; the fixed-state mean tracks the scale of the value function and is the better convergence signal.

Why it is unstable, and a non-standard plot to show it. With $\gamma = 0.95$ a failure k steps ahead lowers the value by only $20 \cdot 0.95^k$: 1.5 at 50 steps, 0.1 at 100 steps. I checked how every failed test episode ended: of the 500 failures of seed 0's last network, 500 ended because the cart left the track and 0 because the pole fell (random play is the opposite: 500 of 500 by pole angle). The same holds for all seeds. The agent balances the pole and drifts sideways until it runs off the track (after 188 steps on average for seed 0's last network).

Figure 5 lines up the last 60 steps before each failure. For CartPole the realized discounted return k steps before a failure has a closed form, $(1-\gamma^{k+1})/(1-\gamma)$, and the measured returns lie exactly on it (a check of the bookkeeping). The network's prediction does not follow: 20 steps before the cart leaves the track it still predicts 19.2 where the truth is 13.2, and at the very last step it predicts 17.6 where the truth is exactly 1. The last networks of the other seeds that fail show the same (prediction at the last step: 19.2, 17.1, 19.4 for seeds 2, 3, 4). So the final value function is nearly blind to the track edge. The edge is only reached after the agent already balances well, terminal transitions there are about one per episode (1252 episodes in 200,000 transitions, about 0.6% of the buffer), and the true value drops from near 20 to 1 over a short range of cart positions. I have not established which of these is the main cause. When the network does represent the edge the agent reaches 500; small later updates can flatten the cliff again.

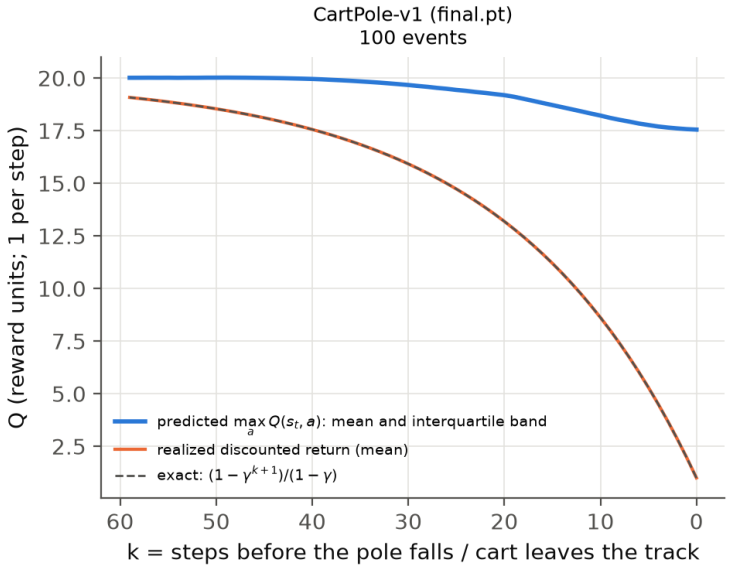


Figure 5: Non-standard plot 1: Q aligned to the next failure (seed 0, `final.pt`, 100 test episodes). k counts steps before the episode ends. Orange: realized discounted return, which coincides with the exact curve $(1-\gamma^{k+1})/(1-\gamma)$ (dashed). Blue: what the network predicts in those states.

I tried four changes that might remove the instability, each over 5 seeds (Figure 6, lower rows): clipping the TD target at the known ceiling of 20, Polyak target updates, a dueling head, and the MSE loss. None helped, and clipping at 20 made it worse ($p = 0.04$ against the default). This tells me the small overestimation above 20 is not the cause. What does fit the explanation is the γ sweep in Section 2.4: the share of validations that are solved rises monotonically with γ.

2.3 Edge cases and how each was handled

Edge case	Handling	Verified by
The 500-step limit is a truncation, not a failure	`cut = 0`, the true 501st observation is stored, the target bootstraps	V4: on an MDP that only ever truncates, DQN matches exact Q^* to 0.4%; the buggy variant is off by 40%. V2(c): no `cut` on truncated episodes
Real termination (pole angle or cart position)	`cut = 1`, target is exactly $r = 1$	V1: terminal target = 1.000000. Ablation: without the mask Q diverges and the reward is 9.5
Last observation of an episode must not be used as a state, and must not be lost	extra non-sampleable buffer slot	V2: 0 draws on such slots; next state always step $t+1$ of the same episode
Q-values above the possible maximum	not prevented; measured against the ceiling 20	Figure 1; 72% of episodes exceed it by at most 24.1 − 20
Unstable training: which network is “the trained network”?	100-episode validation every 5,000 steps on separate seeds; report `best.pt` and `final.pt`	table above; test seeds disjoint from validation and training seeds

Edge case	Handling	Verified by
Evaluation of a truncated episode's return	realized returns within 90 steps of a truncation are dropped ($\gamma^{90} < 0.01$)	Figure 5: realized curve equals the closed form
Ties and near-ties between the two actions	`argmax` (first index); no special handling	action gaps are tiny in safe states by construction, which is the instability discussed above
Restart in the middle of training	atomic checkpoints, logs truncated to the checkpoint, running episodes invalidated in the buffer	V6
No GPU	training stops with an error unless `--allow-cpu`	run on laptop without the flag

2.4 Validation experiments specific to CartPole

Ablations: is every component doing what the theory says? `python -m dqn.ablate --group ablation --seeds 5` trains the default agent with exactly one component changed, 5 seeds each, 200,000 steps. I wrote the predictions before running.

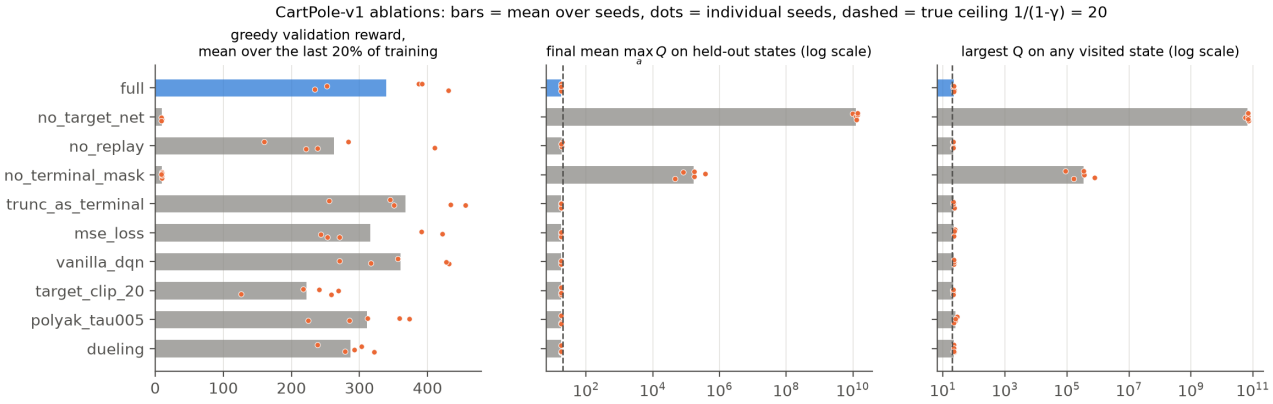


Figure 6: Ablations on CartPole, 5 seeds each. Left: greedy validation reward averaged over the last 20% of training. Middle and right: learned Q against the true ceiling of 20 (log scale). The last three rows are the attempted fixes discussed above.

Variant	Prediction	Validation reward (late mean ± sd over seeds)	Held-out Q at the end	Welch p vs full
full agent	—	339 ± 89	18.1	—
no target network	unstable, Q may diverge	9.4 ± 0.2	$1.2e+10$	0.001
no terminal mask	Q → 20 everywhere, policy random	9.5 ± 0.1	$1.7e+05$	0.001
no replay (buffer = one batch)	fails	263 ± 94	18.5	0.22

Variant	Prediction	Validation reward (late mean ± sd over seeds)	Held-out Q at the end	Welch p vs full
truncation as terminal	small effect at $\gamma = 0.95$	368 ± 80	18.1	0.61
MSE instead of Huber	similar (TD errors are mostly below 1)	316 ± 84	18.0	0.68
vanilla DQN target	more overestimation	360 ± 70	18.2	0.69

What I take from this:

- **Target network and terminal mask are necessary, and the failure is the predicted kind.** Without the target network Q grows to 10^{10} in every seed and the policy is worse than random (9.4 steps; random is 22.4). Without the terminal mask the agent has no information that falling is bad, and it also ends at 9.5 steps.
- **One prediction was only half right.** Without the mask I expected Q to settle at 20, the fixed point of $Q = 1 + 0.95\,Q$. It went to about 10^5 instead. The terminal transitions are the only targets that do not depend on the network's own output; without them nothing anchors the scale, and bootstrapping with function approximation runs away. So the mask matters for stability as well as for correctness.
- **One prediction was wrong.** I expected the agent without replay (a buffer of one batch, the 128 most recent transitions) to fail. It is worse on average (263 vs 339), but with 5 seeds the difference is not significant ($p = 0.22$) and every seed still reached 500 at some point. CartPole with a 4-dimensional state is too easy to show the value of replay; I would not generalize this to MsPacman.
- **Truncation-as-terminal, MSE and the vanilla target make no measurable difference here** ($p \geq 0.6$). For truncation this agrees with the prediction: a wrongly cut bootstrap at step 500 changes values only within about $1/(1-\gamma) = 20$ steps of the limit. The tabular test V4, with a 20-step limit, is where this bug becomes large. With seed-to-seed standard deviations of 70–95 these comparisons can only detect large effects, and I do not claim the variants are equivalent.

Changing γ and checking the predicted ceiling. `python -m dqn.ablate --group gamma --seeds 3`. Prediction: the learned values scale as $1/(1-\gamma)$, and, if my explanation of the instability is right, control should get more reliable as γ grows.

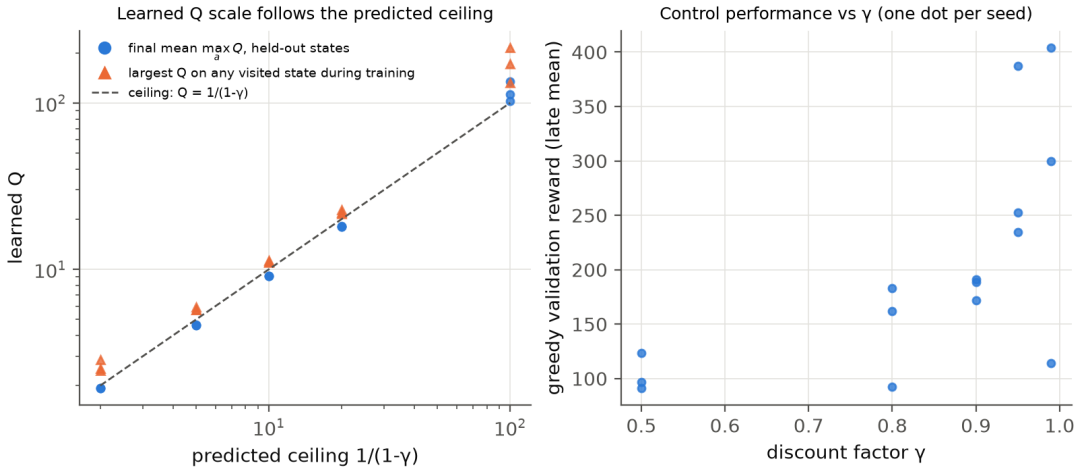


Figure 7: γ **sweep on CartPole**, 3 seeds per value. Left: learned Q against the predicted ceiling $1/(1-\gamma)$ on log axes. Right: late validation reward.

γ	Ceiling $1/(1-\gamma)$	Held-out Q at the end	Largest Q on a visited state	Validations with mean ≥ 475	Late validation reward
0.5	2	1.93	2.62	0%	104
0.8	5	4.62	5.77	1%	146
0.9	10	9.08	11.14	6%	184
0.95	20	18.10	22.10	12%	291
0.99	100	116.6	172.6	32%	273

For $\gamma \leq 0.95$ the held-out value sits at 90–97% of the ceiling, and the largest value seen overshoots it by roughly 10–40%. The share of solved validations rises monotonically with γ, as predicted. At $\gamma = 0.99$ the held-out mean itself (117) is above the ceiling of 100, so overestimation grows with the horizon; the hyperparameters were tuned for 0.95 and I did not retune them.

Random-policy baseline and significance are in the Results above; **the first-10-steps trace** is V1 in Section 4.

3 MsPacman-v5 ($\gamma = 0.99$)

(to be written)

4 Validation of the algorithm itself

The tests in this section do not depend on the game. They run the same agent, buffer and training loop on problems where the right answer is known. Run them with `python -m dqn.validate --allow-cpu` (about 6 minutes on a laptop CPU); each test writes its prediction and measurements to `results/validation/<test>.txt`.

4.1 V1 — The first 10 steps by hand (`trace`)

What it does. It plays CartPole with random actions from seed 0 until the pole falls (18 steps), stores the transitions in the real replay buffer, and asks the agent for the TD targets. The target network is perturbed first so that it differs from the online network. Separately, the test copies the weights out of both networks and recomputes every target with plain numpy: forward pass, arg max from the online network, value from the target network, discount, terminal mask.

Prediction. The two computations agree to float precision; the terminal transition has target exactly $r = 1$; a gradient step moves the online weights and leaves the target weights untouched; after a sync the two are identical.

Result. The maximum difference over all 18 targets is $7.7e-08$. The terminal target is 1.000000. The target weights moved by 0 during the gradient step and differ from the online weights by 0 after the sync. The first rows of the trace:

t	state $(x, \dot{x}, \theta, \dot{\theta})$	a	r	cut	$Q_\theta(s', \cdot)$	$Q_{\bar{\theta}}(s', \cdot)$	y (numpy)	y (agent)
0	(+0.014, −0.023, −0.046, −0.048)	1	1	0	[+0.015, −0.062]	[+0.087, +0.028]	1.08224	1.08224
1	(+0.013, +0.173, −0.047, −0.355)	1	1	0	[+0.018, −0.071]	[+0.064, −0.027]	1.06035	1.06035
8	(+0.042, −0.404, −0.111, +0.334)	0	1	0	[−0.019, −0.025]	[+0.031, +0.002]	1.02934	1.02934
17	(+0.048, +0.974, −0.190, −2.007)	1	1	**1**	[+0.080, −0.086]	[+0.171, −0.247]	1.00000	1.00000

Row 0 can be checked by eye: the online network prefers action 0 in s' ($+0.015 > -0.062$), the target network's value for action 0 is 0.087, and $1 + 0.95 \cdot 0.087 = 1.0822$. In row 8 the online network's two values are close (-0.019 vs -0.025) and it picks action 0; the target is $1 + 0.95 \cdot 0.031 = 1.0293$, which uses the online network's choice and not the target network's own maximum. In row 17 the pole angle is -0.19 rad at s and beyond the 0.209 rad limit at s', so the bootstrap is cut. The complete table is in `results/validation/trace.txt`.

4.2 V2 — Replay buffer: uniform sampling and no stack across episodes (`replay`)

What it does. It fills a small buffer (4 environments, stack 4, 300 slots each) with 20,000 synthetic steps so the ring wraps about 17 times. Every frame encodes (episode id, step), so any mix-up is visible. Episodes have random lengths from 1 to 39 steps. Odd episodes are "truncated", even ones "terminated".

Predictions and results.

Check	Predicted	Measured
Draws that land on an incomplete transition (episode-final slot, newest slot, or a stack partly overwritten)	0	0 of 400,000
Chi-square of the 400,000 draws against the uniform distribution over the 1130 sampleable transitions	p not small	$\chi^2 = 1184$, $p = 0.12$
Frames from a different episode inside a sampled state or next state	0	0 of 800,000 frames
Wrong step index inside a stack (first frame must be repeated at the start of an episode)	0	0
Sampled state differs from the state the acting code saw at that step	0	0 of 10,000
`cut = 1` on a truncated episode	0	0

4.3 V3 — Overfitting one batch (`overfit`)

Prediction. With a frozen target network and a fixed batch of 64 transitions, the targets are constants and the network can memorize them, so the loss must go to about zero. If it does not, the gradient or the indexing of $Q(s, a)$ is wrong.

Result. The MSE falls from 1.067 to $3.4e-07$ in 3,000 steps, and the largest remaining error is $1.8e-03$.

A prediction that was wrong. My first version of this test used the Double DQN target and stalled at a loss of 7×10^{-3}. The targets were not constant: the online network chooses a', so as it is fitted the choice flips between two target-network values. The test now uses the plain max target, where the targets really are fixed. This is not a bug, but it reminded me that with Double DQN the regression target moves even between target syncs.

4.4 V4 — A tabular MDP with exact Q^*, and the truncation bug (`shuttle`)

What it does. I took the shuttle-dispatch MDP from Assignment 1 (here with 21 states, capacity $K = 5$, dispatch cost 20, holding cost 2, $\gamma = 0.95$), whose exact Q^* I get from my Assignment 1 value-iteration code. I wrapped it as an environment with one-hot states, random start states, **no terminal states**, and a time-limit truncation every 20 steps. Then I trained the unchanged DQN on it (40,000 steps, 3 seeds, rewards scaled by 0.02) and compared the learned table with Q^*.

Prediction 1. With the truncation handled correctly, DQN converges to Q^*.

Prediction 2. If the truncation is treated as a termination, one transition in 20 loses its bootstrap. The fixed point is then the Q^* of the same MDP with $\gamma_{\text{eff}} = 0.95 \cdot (1 - \frac{1}{20}) = 0.9025$, whose values are much smaller in magnitude (mean -208.7 instead of -363.4).

Variant (3 seeds each)	mean Q	error vs Q^*	error vs $Q^*_{\gamma_{\text{eff}}}$	greedy policy $= \pi^*$
exact Q^* (value iteration)	-363.4	—	—	—
correct handling, Huber	-362.4	**0.4%**	74%	100% of states
truncation as terminal, MSE	-218.3	40%	**8.1%**	100%
truncation as terminal, Huber	-310.1	15%	49%	100%

(Errors are mean absolute errors over all 42 state–action pairs, relative to the mean magnitude of the reference.)

Result for prediction 1. The correct agent matches Q^* to 0.4% and its greedy policy equals the optimal threshold policy in every state, for all three seeds. Since this environment only ever ends by truncation, this is direct evidence that bootstrapping through a time limit works.

Prediction 2 was wrong at first, and the reason is worth knowing. With my default Huber loss the buggy agent landed at a mean of -310, between the two exact answers, and not at the predicted -209. My derivation assumed the network learns the *mean* of its targets. For a given (s, a) the buggy targets are bimodal: 95% are about -6 (scaled units, with bootstrap) and 5% are about -0.3 (no bootstrap). These are more than the Huber threshold of 1 apart, so the 5% are treated like outliers and pull with a bounded force. I then predicted that the MSE loss would follow my original derivation, and it does: mean -218.3, 8.1% from $Q^*_{\gamma_{\text{eff}}}$. (The remaining gap is expected, because the state distribution at step 20 is not identical to the overall one.)

Two things follow. First, the truncation bug is not cosmetic: it changes the learned values by 15%–40% here. Second, the Huber loss changes *what DQN converges to* whenever targets for the same (s, a) differ by more than 1. This is one more reason to keep rewards (and therefore TD errors) small, which is what reward clipping does in MsPacman.

4.5 V5 — Constant-reward and zero-reward controls (`controls`)

Prediction. If every step pays 1 and nothing terminates, $Q(s, a) = 1/(1-\gamma)$ for every state and action, whatever the dynamics are. With zero reward, $Q \equiv 0$. Any leak of the time limit into the target, a wrong discount, or an off-by-one in the reward would show up as a different constant.

Reward	γ	Predicted Q	Learned Q over all 42 (s, a): min / mean / max
1	0.5	2	2.0000 / 2.0000 / 2.0000
1	0.9	10	10.0011 / 10.0013 / 10.0016
1	0.95	20	19.9992 / 20.0038 / 20.0078
0	0.95	0	-0.0001 / -0.0001 / -0.0001

The largest deviation from the prediction is 0.0078 (at $\gamma = 0.95$, 0.04% of the value).

4.6 V6 — Stop and restart (`resume`)

What it does. It trains CartPole for 3,000 steps, starts the same command again with a 6,000-step budget, and inspects the logs.

Result. The run continues from the checkpoint (step 3,000). The episode log has 176 rows with 0 duplicates and contiguous ids, and the held-out log is strictly increasing. Episodes that were running when the process stopped cannot be continued, so after a restart their newest transition is marked non-sampleable; otherwise it would be paired with the first frame of a new episode.

A prediction that was wrong. I predicted exactly 5,000 gradient updates (6,000 steps minus a 1,000-step warm-up). The run made 5041. The warm-up is counted in buffer *slots*, and every finished episode uses one extra slot for its final frame, so learning starts at step 960. The test now derives the expected count from the episode log and it matches. Nothing was broken, but the hyperparameter table above says "buffer slots" for this reason.